Kleisli Composition

Kleisli · Compose parseInt → enrichUser → authorize · Build with Waqar

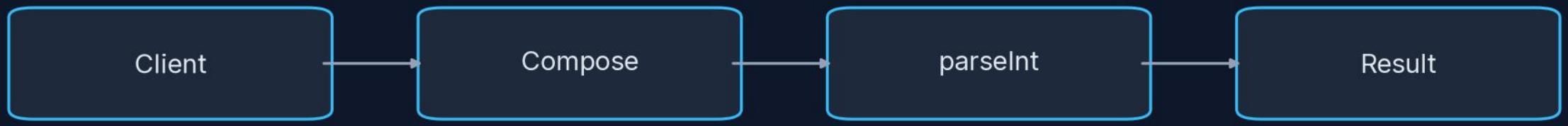
Today's Hook

- Problem: Compose parseInt → enrichUser → authorize
- Pattern: Kleisli — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Compose parseInt → enrichUser → authorize
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Kleisli: Compose parseInt → enrichUser → authorize

Real-World Scenario

- Running example: Compose parseInt → enrichUser → authorize
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.functional.kleisli.problem;

import java.util.*;

public final class NaiveAuth {
  public static String auth(String raw){ int id=Integer.parseInt(raw); return id>0? "allow:"+id: "deny"; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.functional.kleisli.problem;

import java.util.*;
import java.util.function.*;

final class Kleisli<A,B> {
  private final Function<A, Optional<B>> run;
  Kleisli(Function<A, Optional<B>> run){ this.run=run; }
  static <A,B> Kleisli<A,B> of(Function<A, Optional<B>> f){ return new Kleisli<>(f); }
  <C> Kleisli<A,C> andThen(Kleisli<B,C> next){ return new Kleisli<>(a -> run.apply(a).flatMap(next.run)); }
  Optional<B> apply(A a){ return run.apply(a); }
}
final class AuthPipe {
  static final Kleisli<String,Integer> parse = Kleisli.of(s -> { try { return Optional.of(Integer.parseInt(s)); } catch(Exception e){ return Optional.empty(); } });
  static final Kleisli<Integer,Integer> enrich = Kleisli.of(id -> id>0? Optional.of(id+1000): Optional.empty());
  static final Kleisli<Integer,String> authorize = Kleisli.of(id -> Optional.of("allow:"+id));
  static final Kleisli<String,String> pipeline = parse.andThen(enrich).andThen(authorize);
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.kleisli.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Kleisli: parse → enrich → authorize ===");
    System.out.println("--- naive ---"); System.out.println(NaiveAuth.auth("7"));
System.out.println("--- kleisli ---");
System.out.println(AuthPipe.pipeline.apply("7"));
System.out.println(AuthPipe.pipeline.apply("x"));
System.out.println(AuthPipe.pipeline.apply("-1"));
  }
}
```

Walkthrough Output

```
=== Kleisli: parse → enrich → authorize ===
--- naive ---
allow:7
--- kleisli ---
Optional[allow:1007]
Optional.empty
Optional.empty
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Kleisli.

Related Patterns

- Contrast Kleisli with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Kleisli → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.